

Week 4:

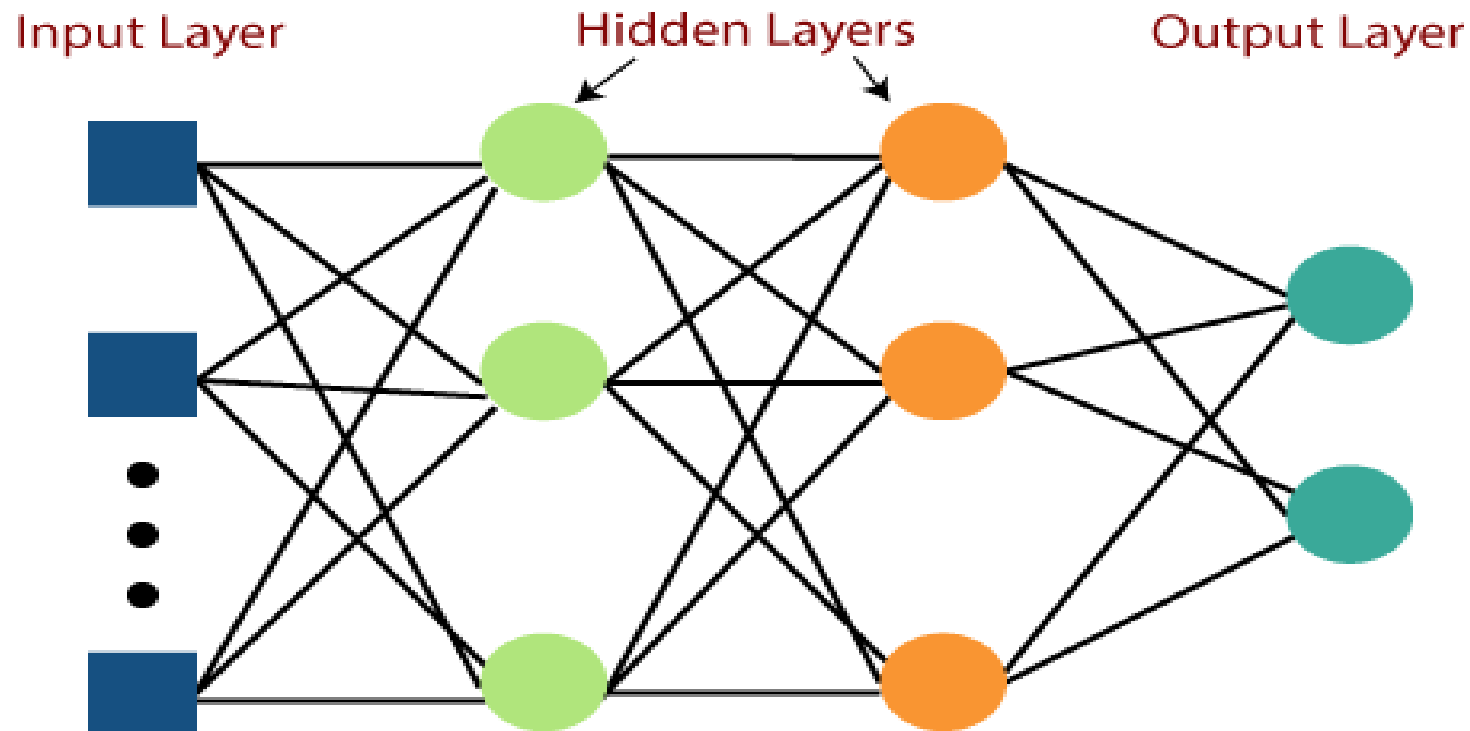
**Implementation of Multilayer Neural Network using Keras
and Data Augmentation on MNIST dataset**

Aim of the Program:

- ❖ To develop and train a Convolutional Neural Network (CNN) using the Keras library for handwritten digit classification on the MNIST dataset, incorporating data augmentation techniques to improve model generalization and performance.**

❖ Multilayer Neural Network (MLN)

- ❖ A multilayer neural network (MLN) is a type of artificial neural network that has more than one layer of neurons. MLNs are typically **feed-forward networks**, which means that the information flows from the input layer to the output layer in one direction only.
- ❖ MLNs are able to learn complex patterns in data by combining the outputs of neurons in previous layers. This allows them to solve a wide range of problems, including image classification, natural language processing, and machine translation.



❖ **How does it work?**

- ❖ **Input Layer:** Receives the input data and passes it to the first hidden layer.
- ❖ **Hidden Layers:** Each neuron in a hidden layer takes a weighted sum of the outputs from the previous layer, applies an activation function, and passes the result to the next layer. This process continues through all hidden layers.
- ❖ **Output Layer:** The final layer produces the output, often representing a prediction or classification.

❑ **Advantages of multilayer neural networks:**

- ❖ MLNs are able to learn complex patterns in data that are difficult to learn with other types of machine learning models.
- ❖ MLNs are versatile and can be used to solve a wide range of problems.
- ❖ MLNs have been shown to achieve state-of-the-art results on many machine learning benchmarks.

❖ MNIST Dataset

- ❖ we create a multilayer neural network for **classifying the MNIST dataset** (Modified National Institute of Standards and Technology), which contains **images of handwritten digits**.
- ❖ We preprocess the dataset, apply data augmentation techniques to improve model generalization, define a convolutional neural network architecture, compile the model with appropriate settings, and train it using the augmented training data.
- ❖ After training, we evaluate the model's accuracy on the test dataset to assess its performance.



❖ **The program implements a Convolutional Neural Network (CNN) using Keras to classify images from the MNIST dataset (handwritten digits). It also utilizes data augmentation to improve the model's generalization ability.**

1. Installing necessary libraries.
2. Loading and preprocessing the MNIST dataset.
3. Setting up data augmentation.
4. Building the neural network model.
5. Compiling and training the model.
6. Plotting training history.
7. Making predictions and generating a classification report.
8. Visualizing predictions.

Step 1: Install Required Libraries

1. `pip install tensorflow keras numpy matplotlib scikit-learn`

Step 2: Import Libraries

1. `import numpy as np`
2. `import tensorflow as tf`
3. `from tensorflow import keras`
4. `from tensorflow.keras.models import Sequential`
5. `from tensorflow.keras.layers import Dense, Flatten, Conv2D, MaxPooling2D`
6. `from tensorflow.keras.preprocessing.image import ImageDataGenerator`
7. `from keras.datasets import mnist`
8. `import matplotlib.pyplot as plt`
9. `from sklearn.metrics import classification_report`

Step 3: Load and Preprocess the MNIST Dataset

Load the MNIST dataset

1. (train_images, train_labels), (test_images, test_labels) = mnist.load_data()

Normalize and reshape the images

1. train_images = train_images.reshape((60000, 28, 28, 1)).astype('float32') / 255

2. test_images = test_images.reshape((10000, 28, 28, 1)).astype('float32') / 255

One-hot encode the labels

1. train_labels = keras.utils.to_categorical(train_labels, 10)

2. test_labels = keras.utils.to_categorical(test_labels, 10)

Step 4: Set Up Data Augmentation

```
1. datagen = ImageDataGenerator(  
2.     rotation_range=40,  
3.     width_shift_range=0.2,  
4.     height_shift_range=0.2,  
5.     shear_range=0.2,  
6.     zoom_range=0.2,  
7.     horizontal_flip=True,  
8.     fill_mode='nearest'  
9. )
```

Fit the data generator to the training images

```
1. datagen.fit(train_images)
```

Step 5: Build the Neural Network Model

1. `model = Sequential()`
2. `model.add(Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)))`
3. `model.add(MaxPooling2D((2, 2)))`
4. `model.add(Conv2D(64, (3, 3), activation='relu'))`
5. `model.add(MaxPooling2D((2, 2)))`
6. `model.add(Conv2D(64, (3, 3), activation='relu'))`
7. `model.add(Flatten())`
8. `model.add(Dense(64, activation='relu'))`
9. `model.add(Dense(10, activation='softmax'))`

Display the model's architecture

1. `model.summary()`

```
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
=====		
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
conv2d_2 (Conv2D)	(None, 3, 3, 64)	36928
flatten (Flatten)	(None, 576)	0
dense (Dense)	(None, 64)	36928
dense_1 (Dense)	(None, 10)	650
=====		
Total params: 93322 (364.54 KB)		
Trainable params: 93322 (364.54 KB)		
Non-trainable params: 0 (0.00 Byte)		
=====		

Code Explanation:

- ❖ `Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1))`
- ❖ Input image: 28x28 pixels, 1 channel (black & white)
- ❖ Applies 32 filters of size 3x3 - dimensions of the filter (or kernel), Common filter sizes are 3x3, 5x5, or 7x7, and they are specified as (height, width)
- ❖ Removes 1 pixel border (no padding) → becomes 26x26
- ❖ **Output: (26, 26, 32)**
- ❖ Output = $((28 - 3) / 1) + 1$
= $(25 / 1) + 1$
= $25 + 1$
= 26
- ❖ So the output shape becomes: 26×26 (for each filter)

❖ MaxPooling2D((2, 2))

- ❖ Takes 2x2 blocks → keeps the max value
- ❖ Reduces size by half: 26x26 → 13x13
- ❖ Output: (13, 13, 32)

❖ Conv2D(64, (3, 3), activation='relu')

- ❖ 64 filters: The model will learn 64 different patterns (edges, textures, etc.)
- ❖ Kernel size (3,3): Each filter is of size 3×3

❖ Input = (13, 13, 32)

❖ **This means:**

- ❖ Width = 13
- ❖ Height = 13
- ❖ Depth (Channels) = 32
- ❖ Now this Conv2D layer applies 64 filters to this input

❖ **Output Width = ((Input Width - Kernel Width) / Stride) + 1**

❖ Input Width = 13

❖ Kernel Width = 3

❖ Stride = 1 (default)

❖ Padding = 'valid' (no padding)

❖ Output Width = $((13 - 3) / 1) + 1$
 $= (10 / 1) + 1$
 $= 11$

❖ So, both height and width reduce from 13 $\rightarrow$ 11

❖ **(11, 11, 64)**

❖ 11x11 $\rightarrow$ because of size reduction

❖ 64 $\rightarrow$ because we used 64 filters (each filter creates 1 output feature map)

❖ Let's also calculate how many trainable parameters this layer has:

❖ **Formula:**

❖ **$(3 \times 3 \times \text{Previous Depth} + 1) \times \text{Number of Filters}$**

❖ $3 \times 3 = 9$

❖ Previous depth = 32

❖ So: $(9 \times 32 + 1) \times 64 = (288 + 1) \times 64 = 289 \times 64 = \mathbf{18,496}$

❖ Input: $13 \times 13 \times 32$

❖ Applies: 64 filters of 3×3

❖ Output becomes: $11 \times 11 \times 64$

❖ Because each 3×3 filter slides across 11×11 positions

❖ No padding $\rightarrow$ so size shrinks

❖ Total trainable parameters: 18,496

❖ Flatten()

- ❖ Flattens 3D image (3 x 3 x 64 = 576 values)
- ❖ Converts to 1D: (576,)
- ❖ Flatten() is a layer that takes a multi-dimensional array (like a cube or grid of values) and converts it into a single long row (a 1D array).

❖ Dense(64, activation='relu')

- ❖ Fully connected layer: takes 576 inputs, gives 64 outputs
- ❖ Output: (64,)

- ❖ It takes 576 inputs (from the Flatten() layer)

- ❖ It has 64 neurons (units)

- ❖ Each neuron in this layer:

- ❖ Connects to all 576 inputs

- ❖ Has 576 weights (one per input)

- ❖ Has 1 bias (extra value added)

So for 1 neuron:

- It needs to **learn 576 weights**

- And **1 bias**

- Total: **577 parameters** per neuron

577 parameters × 64 neurons = 36,928 parameters

CNN Model Layer Summary with Input/Output Shapes and Parameter Calculation

Step	Layer	Input Shape	Operation Description	Output Shape	Parameter Calculation	Parameters
1	Conv2D(32, (3, 3))	(28, 28, 1)	32 filters of 3×3 on 1 channel	(26, 26, 32)	$(3 \times 3 \times 1 + 1) \times 32 = 10 \times 32$	320
2	MaxPooling2D(2, 2)	(26, 26, 32)	2×2 pooling	(13, 13, 32)	No parameters	0
3	Conv2D(64, (3, 3))	(13, 13, 32)	64 filters of 3×3 on 32 channels	(11, 11, 64)	$(3 \times 3 \times 32 + 1) \times 64 = (289 + 1) \times 64 = 18,496$	18,496
4	MaxPooling2D(2, 2)	(11, 11, 64)	2×2 pooling	(5, 5, 64)	No parameters	0

Step	Layer	Input Shape	Operation Description	Output Shape	Parameter Calculation	Parameters
5	Conv2D(64, (3, 3))	(5, 5, 64)	64 filters of 3×3 on 64 channels	(3, 3, 64)	$(3 \times 3 \times 64 + 1) \times 64 = (577) \times 64 = 36,928$	36,928
6	Flatten()	(3, 3, 64)	Flattens 3D to 1D: $3 \times 3 \times 64 = 576$	(576,)	No parameters	0
7	Dense(64)	(576,)	Fully connected: 64 outputs	(64,)	$(576 + 1) \times 64 = 577 \times 64 = 36,928$	36,928
8	Dense(10)	(64,)	Final output layer with 10 classes	(10,)	$(64 + 1) \times 10 = 65 \times 10 = 650$	650

Total Parameters = $320 + 0 + 18,496 + 0 + 36,928 + 0 + 36,928 + 650 = 93,322$

Step 6: Compile and Train the Model

Compile the model

- 1. model.compile(optimizer='adam',**
- 2. loss='categorical_crossentropy',**
- 3. metrics=['accuracy'])**

Train the model with data augmentation

- 1. history = model.fit(datagen.flow(train_images, train_labels, batch_size=32),**
- 2. steps_per_epoch=len(train_images) / 32,**
- 3. epochs=3,**
- 4. validation_data=(test_images, test_labels))**

Epoch 1/3

1875/1875 [=====] - 99s 51ms/step - loss: 0.9955 - accuracy: 0.6594 - val_loss: 0.2708 - val_accuracy: 0.9161

Epoch 2/3

1875/1875 [=====] - 112s 60ms/step - loss: 0.4747 - accuracy: 0.8462 - val_loss: 0.1854 - val_accuracy: 0.9440

Epoch 3/3

1875/1875 [=====] - 93s 50ms/step - loss: 0.3620 - accuracy: 0.8847 - val_loss: 0.1547 - val_accuracy: 0.9505

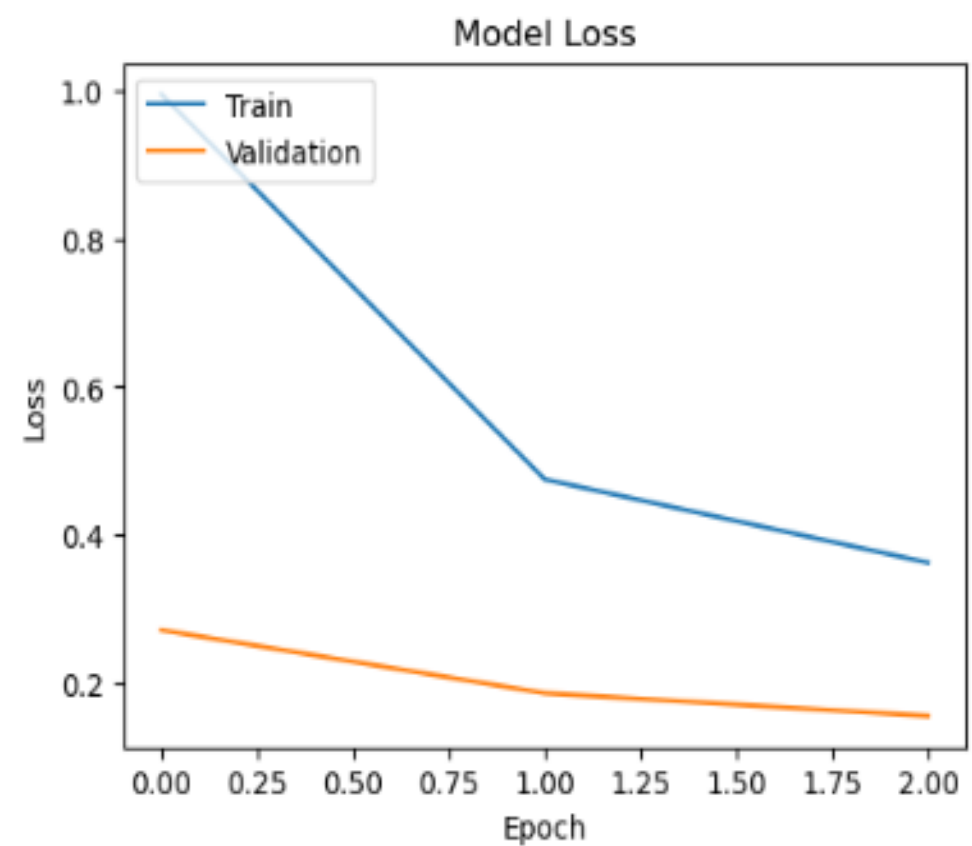
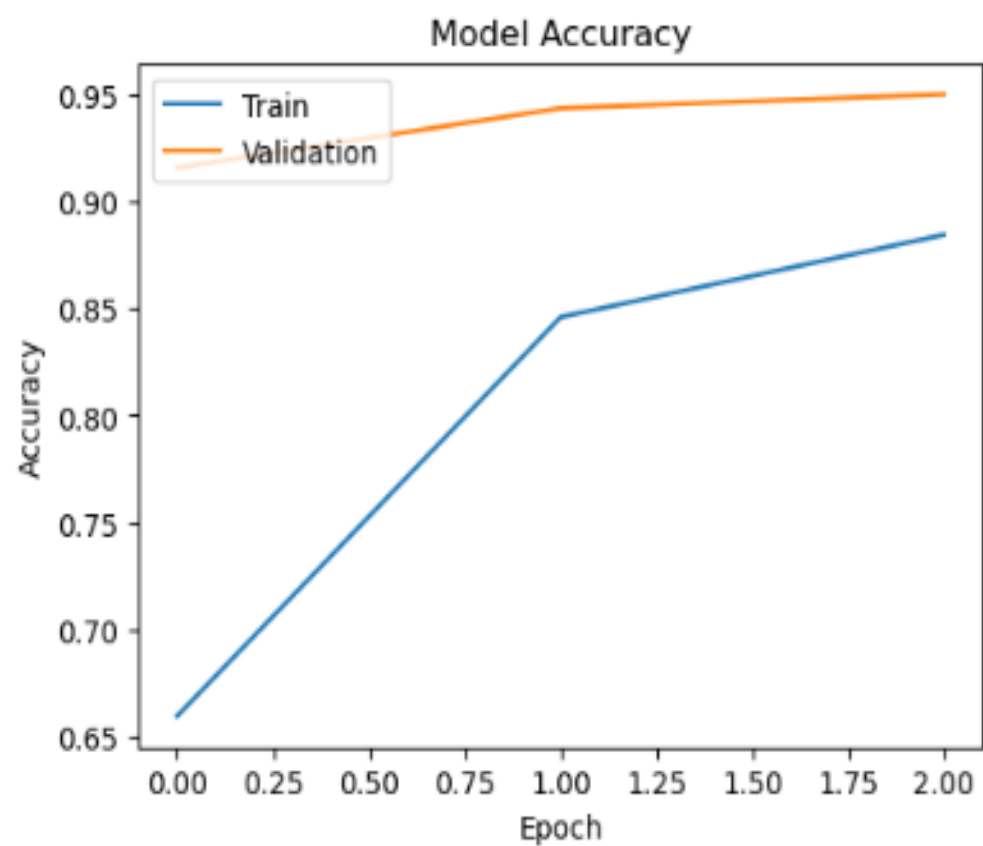
Step 7: Plot Training History

Plot training & validation accuracy values

1. `plt.figure(figsize=(12, 4))`
2. `plt.subplot(1, 2, 1)`
3. `plt.plot(history.history['accuracy'])`
4. `plt.plot(history.history['val_accuracy'])`
5. `plt.title('Model Accuracy')`
6. `plt.ylabel('Accuracy')`
7. `plt.xlabel('Epoch')`
8. `plt.legend(['Train', 'Validation'], loc='upper left')`

Plot training & validation loss values

1. `plt.subplot(1, 2, 2)`
2. `plt.plot(history.history['loss'])`
3. `plt.plot(history.history['val_loss'])`
4. `plt.title('Model Loss')`
5. `plt.ylabel('Loss')`
6. `plt.xlabel('Epoch')`
7. `plt.legend(['Train', 'Validation'], loc='upper left')`
8. `plt.show()`



Step 8: Make Predictions and Generate a Classification Report

Make predictions

- 1. `predictions = model.predict(test_images)`**
- 2. `predicted_classes = np.argmax(predictions, axis=1)`**
- 3. `true_classes = np.argmax(test_labels, axis=1)`**

Generate a classification report

- 1. `print(classification_report(true_classes, predicted_classes))`**

313/313 [=====] - 4s 12ms/step

	precision	recall	f1-score	support
0	0.98	0.98	0.98	980
1	1.00	0.97	0.98	1135
2	0.86	0.90	0.88	1032
3	0.96	0.97	0.96	1010
4	0.98	0.97	0.97	982
5	0.91	0.92	0.92	892
6	0.92	0.97	0.95	958
7	0.97	0.90	0.94	1028
8	0.98	0.94	0.96	974
9	0.95	0.97	0.96	1009
accuracy			0.95	10000
macro avg	0.95	0.95	0.95	10000
weighted avg	0.95	0.95	0.95	10000

Step 9: Visualize Some Predictions

Display a few test images with their predicted and true labels

- 1. `num_images = 5`**
- 2. `plt.figure(figsize=(10, 10))`**
- 3. `for i in range(num_images):`**
- 4. `plt.subplot(1, num_images, i + 1)`**
- 5. `plt.imshow(test_images[i].reshape(28, 28), cmap='gray')`**
- 6. `plt.title(f"Pred: {predicted_classes[i]}\nTrue: {true_classes[i]}")`**
- 7. `plt.axis('off')`**
- 8. `plt.show()`**

Pred: 7
True: 7



Pred: 2
True: 2



Pred: 1
True: 1



Pred: 0
True: 0



Pred: 4
True: 4



Step 10: Evaluate the Model on Test Data

Evaluate the model on the test data

1. `test_loss, test_acc = model.evaluate(test_images, test_labels)`
2. `print(f'Test accuracy: {test_acc:.4f}')`

```
313/313 [=====] - 4s 12ms/step - loss: 0.1547 - accuracy: 0.9505  
Test accuracy: 0.9505
```

Summary:

- ❖ This Python program implements a Convolutional Neural Network (CNN) using Keras to classify images from the MNIST dataset, utilizing data augmentation to enhance model performance. The dataset is preprocessed by normalizing and reshaping the images and one-hot encoding the labels. Data augmentation is applied to the training images using random transformations, such as rotation and zoom, to help the model generalize better. The CNN, consisting of convolutional, pooling, and dense layers, is compiled with the Adam optimizer and trained for three epochs. The model's performance is evaluated by plotting training history, generating a classification report, and visualizing predictions, achieving a final accuracy score on the test dataset.